Data Structures

HomeWork Set -6

Name : V.Rahul

Roll number : 2025115120

1. Simulate Undo and Redo in MS Word using a Doubly Linked List

```
#include  <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node {
    char text[100];
    struct Node *prev;
    struct Node *next;
};

struct Node *current = NULL;

struct Node *createNode(char text[]) {
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));

    strcpy(newNode->text, text);
    newNode->prev = NULL;
    newNode->next = NULL;

    return newNode;
}

void writeText(char text[]) {
    struct Node *newNode = createNode(text);

    if (current == NULL) {
        current = newNode;
        return;
    }

    // Remove redo history after a new edit
    struct Node *temp = current->next;

    while (temp != NULL) {
        struct Node *nextNode = temp->next;
        free(temp);
        temp = nextNode;
    }
```

```
current->next = newNode;
```

```
    newNode->prev = current;
    current = newNode;
}

void undo() {
    if (current == NULL || current->prev == NULL) {
        printf("Nothing to undo!\n");
    } else {
        current = current->prev;
        printf("Undo: %s\n", current->text);
    }
}

void redo() {
    if (current == NULL || current->next == NULL) {
        printf("Nothing to redo!\n");
    } else {
        current = current->next;
        printf("Redo: %s\n", current->text);
    }
}

void display() {
    if (current == NULL)
        printf("Document is empty\n");
    else
        printf("Current text: %s\n", current->text);
}

int main() {
    writeText("Hello");
    display();

    writeText("Hello World");
    display();

    writeText("Hello World!");
    display();

    undo();
    undo();

    redo();

    writeText("Hello C");

    display();

    redo();
```

```
    return 0;
}
```

Output :

```
Current text: Hello
Current text: Hello World
Current text: Hello World!
Undo: Hello World
Undo: Hello
Redo: Hello World
Current text: Hello C
Nothing to redo!
```

2. Given two sorted lists \(L_1\) and \(L_2\), compute \(L_1 \cap L_2\) and \(L_1 \cup L_2\) using basic list operations.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *createNode(int x) {
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));

    newNode->data = x;
    newNode->next = NULL;
    return newNode;
}

void insertEnd(struct Node **head, int x) {
    struct Node *newNode = createNode(x);

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    struct Node *temp = *head;

    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
```

```
}

void appendUnique(struct Node **head, int x) {
    if (*head != NULL) {
        struct Node *temp = *head;

        while (temp != NULL) {
            if (temp->data == x)
                return;
            temp = temp->next;
        }
    }

    insertEnd(head, x);
}

void findUnionIntersection(
    struct Node *a,
    struct Node *b,
    struct Node **uni,
    struct Node **inter) {

    while (a != NULL && b != NULL) {
        if (a->data < b->data) {
            appendUnique(uni, a->data);
            a = a->next;
        }
        else if (a->data > b->data) {
            appendUnique(uni, b->data);
            b = b->next;
        }
        else {
            appendUnique(uni, a->data);
            appendUnique(inter, a->data);
            a = a->next;
            b = b->next;
        }
    }

    while (a != NULL) {
        appendUnique(uni, a->data);
        a = a->next;
    }

    while (b != NULL) {
        appendUnique(uni, b->data);
        b = b->next;
    }
}
```

```
void display(struct Node *head) {
    while (head != NULL) {
        printf("%d", head->data);
        if (head->next != NULL)
            printf(" -> ");
        head = head->next;
    }
    printf(" -> NULL\n");
}

int main() {
    struct Node *L1 = NULL, *L2 = NULL;
    struct Node *uni = NULL, *inter = NULL;

    int n, m, x;

    printf("Enter number of elements in L1: ");
    scanf("%d", &n);

    printf("Enter sorted elements of L1: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &x);
        insertEnd(&L1, x);
    }

    printf("Enter number of elements in L2: ");
    scanf("%d", &m);

    printf("Enter sorted elements of L2: ");
    for (int i = 0; i < m; i++) {
        scanf("%d", &x);
        insertEnd(&L2, x);
    }

    findUnionIntersection(L1, L2, &uni, &inter);

    printf("Intersection: ");
    display(inter);

    printf("Union: ");
    display(uni);

    return 0;
}
```

Output :

```
Enter number of elements in L1: 4
Enter sorted elements of L1: 1 3 5 7
```

```
Enter number of elements in L2: 4
Enter sorted elements of L2: 2 3 5 8
Intersection: 3 -> 5 -> NULL
Union: 1 -> 2 -> 3 -> 5 -> 7 -> 8 -> NULL
```

3. Swap two elements using a doubly linked list without changing the data.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
};

struct Node *createNode(int x) {
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));

    newNode->data = x;
    newNode->prev = NULL;
    newNode->next = NULL;

    return newNode;
}

void insertEnd(struct Node **head, int x) {
    struct Node *newNode = createNode(x);

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    struct Node *temp = *head;

    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;
}

void swapNodes(struct Node **head, int x, int y) {
    if (x == y)
        return;
```

```
struct Node *a = *head;
struct Node *b = *head;

while (a != NULL && a->data != x)
    a = a->next;

while (b != NULL && b->data != y)
    b = b->next;

if (a == NULL || b == NULL) {
    printf("One or both elements not found\n");
    return;
}

// Ensure a comes before b
if (a->next == b) {
    struct Node *before = a->prev;
    struct Node *after = b->next;

    if (before != NULL)
        before->next = b;
    else
        *head = b;

    b->prev = before;
    b->next = a;

    a->prev = b;
    a->next = after;

    if (after != NULL)
        after->prev = a;
}
else if (b->next == a) {
    swapNodes(head, y, x);
}
else {
    struct Node *ap = a->prev;
    struct Node *an = a->next;
    struct Node *bp = b->prev;
    struct Node *bn = b->next;

    if (ap != NULL)
        ap->next = b;
    else
        *head = b;

    if (an != NULL)
        an->prev = b;
```

```
        if (bp != NULL)
            bp->next = a;
        else
            *head = a;

        if (bn != NULL)
            bn->prev = a;

        a->prev = bp;
        a->next = bn;

        b->prev = ap;
        b->next = an;
    }
}

void display(struct Node *head) {
    while (head != NULL) {
        printf("%d", head->data);

        if (head->next != NULL)
            printf(" <-> ");

        head = head->next;
    }

    printf(" <-> NULL\n");
}

int main() {
    struct Node *head = NULL;
    int n, x, a, b;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &x);
        insertEnd(&head, x);
    }

    printf("Enter two elements to swap: ");
    scanf("%d %d", &a, &b);

    printf("Before swapping: ");
    display(head);

    swapNodes(&head, a, b);
```

```
    printf("After swapping: ");
    display(head);

    return 0;
}
```

Output :

```
Enter number of nodes: 4
Enter elements: 10 20 30 40
Enter two elements to swap: 20 40
Before swapping: 10 <-> 20 <-> 30 <-> 40 <-> NULL
After swapping: 10 <-> 40 <-> 30 <-> 20 <-> NULL
```

4. Write an array implementation of self-adjusting lists.

```
#include <stdio.h>

#define MAX 100

int list[MAX];
int size = 0;

void insertFront(int x) {
    if (size == MAX) {
        printf("List is full!\n");
        return;
    }

    for (int i = size; i > 0; i--) {
        list[i] = list[i - 1];
    }

    list[0] = x;
    size++;
}

void find(int x) {
    int i;

    for (i = 0; i < size; i++) {
        if (list[i] == x)
            break;
    }

    if (i == size) {
        printf("Element not found!\n");
        return;
    }
```

```
    int found = list[i];

    for (int j = i; j > 0; j--) {
        list[j] = list[j - 1];
    }

    list[0] = found;

    printf("Element found and moved to front.\n");
}

void display() {
    printf("List: ");

    for (int i = 0; i < size; i++) {
        printf("%d ", list[i]);
    }

    printf("\n");
}

int main() {
    int choice, x;

    do {
        printf("\n1. Insert at front\n");
        printf("2. Find element\n");
        printf("3. Display list\n");
        printf("4. Exit\n");

        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter element: ");
                scanf("%d", &x);
                insertFront(x);
                break;

            case 2:
                printf("Enter element to find: ");
                scanf("%d", &x);
                find(x);
                break;

            case 3:
                display();
                break;
```

```
            case 4:
                printf("Exiting...\n");
                break;

            default:
                printf("Invalid choice!\n");
        }
    } while (choice != 4);

    return 0;
}
```

Output:

```
 1
10
1
20
1
30
3
2
10
3
4
List: 30 20 10
Element found and moved to front.
List: 10 30 20
Exiting…
```

5. Write a C program to reverse a given doubly linked list.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
};

struct Node *createNode(int x) {
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));

    newNode->data = x;
    newNode->prev = NULL;
```

```
    newNode->next = NULL;

    return newNode;
}

void insertEnd(struct Node **head, int x) {
    struct Node *newNode = createNode(x);

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    struct Node *temp = *head;

    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;
}

void reverse(struct Node **head) {
    struct Node *current = *head;
    struct Node *temp = NULL;

    while (current != NULL) {
        temp = current->prev;
        current->prev = current->next;
        current->next = temp;

        current = current->prev;
    }

    if (temp != NULL)
        *head = temp->prev;
}

void display(struct Node *head) {
    while (head != NULL) {
        printf("%d", head->data);

        if (head->next != NULL)
            printf(" <-> ");

        head = head->next;
    }

    printf(" <-> NULL\n");
}
```

```
int main() {
    struct Node *head = NULL;
    int n, x;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter elements: ");

    for (int i = 0; i < n; i++) {
        scanf("%d", &x);
        insertEnd(&head, x);
    }

    printf("Original list: ");
    display(head);

    reverse(&head);

    printf("Reversed list: ");
    display(head);

    return 0;
}
```

Output:

```
Enter number of nodes: 4
Enter elements: 10 20 30 40
Original list: 10 <-> 20 <-> 30 <-> 40 <-> NULL
Reversed list: 40 <-> 30 <-> 20 <-> 10 <-> NULL
```

6. Write a C program to delete the middle element of a DLL.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
};

struct Node *createNode(int x) {
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));

    newNode->data = x;
```

```
    newNode->prev = NULL;
    newNode->next = NULL;

    return newNode;
}

void insertEnd(struct Node **head, int x) {
    struct Node *newNode = createNode(x);

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    struct Node *temp = *head;

    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;
}

void deleteMiddle(struct Node **head) {
    if (*head == NULL) {
        printf("List is empty!\n");
        return;
    }

    struct Node *slow = *head;
    struct Node *fast = *head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }

    // Remove the middle node
    if (slow->prev != NULL)
        slow->prev->next = slow->next;
    else
        *head = slow->next;

    if (slow->next != NULL)
        slow->next->prev = slow->prev;

    free(slow);
}

void display(struct Node *head) {
```

```
    while (head != NULL) {
        printf("%d", head->data);

        if (head->next != NULL)
            printf(" <-> ");

        head = head->next;
    }

    printf(" <-> NULL\n");
}

int main() {
    struct Node *head = NULL;
    int n, x;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter elements: ");

    for (int i = 0; i < n; i++) {
        scanf("%d", &x);
        insertEnd(&head, x);
    }

    printf("Original list: ");
    display(head);

    deleteMiddle(&head);

    printf("After deleting middle: ");
    display(head);

    return 0;
}
```

Output:

```
Enter number of nodes: 5
Enter elements: 10 20 30 40 50
Original list: 10 <-> 20 <-> 30 <-> 40 <-> 50 <-> NULL
After deleting middle: 10 <-> 20 <-> 40 <-> 50 <-> NULL
```

7. Remove duplicate elements from a doubly linked list

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
};

struct Node *createNode(int x) {
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));

    newNode->data = x;
    newNode->prev = NULL;
    newNode->next = NULL;

    return newNode;
}

void insertEnd(struct Node **head, int x) {
    struct Node *newNode = createNode(x);

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    struct Node *temp = *head;

    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;
}

void display(struct Node *head) {
    while (head != NULL) {
        printf("%d", head->data);

        if (head->next != NULL)
            printf(" <-> ");

        head = head->next;
    }

    printf(" <-> NULL\n");
}

// Part (i): Remove duplicates from a sorted DLL
void removeSortedDuplicates(struct Node *head) {
    struct Node *current = head;
```

```
    while (current != NULL && current->next != NULL) {
        if (current->data == current->next->data) {
            struct Node *duplicate = current->next;

            current->next = duplicate->next;

            if (duplicate->next != NULL)
                duplicate->next->prev = current;

            free(duplicate);
        } else {
            current = current->next;
        }
    }
}

// Part (ii): Remove duplicates from an unsorted DLL
void removeUnsortedDuplicates(struct Node *head) {
    struct Node *current = head;

    while (current != NULL) {
        struct Node *temp = current->next;

        while (temp != NULL) {
            if (temp->data == current->data) {
                struct Node *duplicate = temp;
                temp = temp->next;

                if (duplicate->prev != NULL)
                    duplicate->prev->next = duplicate->next;

                if (duplicate->next != NULL)
                    duplicate->next->prev = duplicate->prev;

                free(duplicate);
            } else {
                temp = temp->next;
            }
        }

        current = current->next;
    }
}

int main() {
    struct Node *sorted = NULL;
    struct Node *unsorted = NULL;

    int n, x;
```

```
    printf("Enter number of sorted-list elements: ");
    scanf("%d", &n);

    printf("Enter sorted elements: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &x);
        insertEnd(&sorted, x);
    }

    printf("Sorted list before: ");
    display(sorted);

    removeSortedDuplicates(sorted);

    printf("Sorted list after: ");
    display(sorted);

    printf("\nEnter number of unsorted-list elements: ");
    scanf("%d", &n);

    printf("Enter unsorted elements: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &x);
        insertEnd(&unsorted, x);
    }

    printf("Unsorted list before: ");
    display(unsorted);

    removeUnsortedDuplicates(unsorted);

    printf("Unsorted list after: ");
    display(unsorted);

    return 0;
}
```

Output :

```
Enter number of sorted-list elements: 6
Enter sorted elements: 10 10 20 30 30 40

Enter number of unsorted-list elements: 5
Enter unsorted elements: 10 30 10 20 30
Sorted list before: 10 <-> 10 <-> 20 <-> 30 <-> 30 <-> 40 <-> NULL
Sorted list after: 10 <-> 20 <-> 30 <-> 40 <-> NULL

Unsorted list before: 10 <-> 30 <-> 10 <-> 20 <-> 30 <-> NULL
Unsorted list after: 10 <-> 30 <-> 20 <-> NULL
```

8. Calculate the frequency count of a given integer in a circular DLL.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
};

struct Node *createNode(int x) {
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));

    newNode->data = x;
    newNode->prev = NULL;
    newNode->next = NULL;

    return newNode;
}

void insertEnd(struct Node **head, int x) {
    struct Node *newNode = createNode(x);

    if (*head == NULL) {
        *head = newNode;
        newNode->next = newNode;
        newNode->prev = newNode;
        return;
    }

    struct Node *last = (*head)->prev;

    newNode->next = *head;
    newNode->prev = last;

    last->next = newNode;
    (*head)->prev = newNode;
}

int frequency(struct Node *head, int key) {
    if (head == NULL)
        return 0;

    int count = 0;
    struct Node *temp = head;

    do {
```

```
        if (temp->data == key)
            count++;

        temp = temp->next;
    } while (temp != head);

    return count;
}

void display(struct Node *head) {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    struct Node *temp = head;

    do {
        printf("%d", temp->data);
        temp = temp->next;

        if (temp != head)
            printf(" <-> ");
    } while (temp != head);

    printf(" (circular)\n");
}

int main() {
    struct Node *head = NULL;
    int n, x, key;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter elements: ");

    for (int i = 0; i < n; i++) {
        scanf("%d", &x);
        insertEnd(&head, x);
    }

    printf("Enter element to count: ");
    scanf("%d", &key);

    printf("Circular DLL: ");
    display(head);

    printf("Frequency of %d: %d\n",
           key, frequency(head, key));
```

```
    return 0;
}
```

Output :

```
Enter number of nodes: 5
Enter elements: 10 20 10 30 10
Enter element to count: 10
Circular DLL: 10 <-> 20 <-> 10 <-> 30 <-> 10 (circular)
Frequency of 10: 3
```

9. Search for a particular element in a binary tree

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left;
    struct Node *right;
};

struct Node *createNode(int x) {
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));

    newNode->data = x;
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}

int search(struct Node *root, int key) {
    if (root == NULL)
        return 0;

    if (root->data == key)
        return 1;

    if (search(root->left, key))
        return 1;

    return search(root->right, key);
}

int main() {
    int key;
```

```
    // Create the binary tree
    struct Node *root = createNode(10);
    root->left = createNode(20);
    root->right = createNode(30);
    root->left->left = createNode(40);
    root->left->right = createNode(50);

    printf("Enter element to search: ");
    scanf("%d", &key);

    if (search(root, key))
        printf("Element found\n");
    else
        printf("Element not found\n");

    return 0;
}
```

Output :

```
Enter element to search: 40
Element found
```

10. Write a C program to calculate and print the height or depth of a tree.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left;
    struct Node *right;
};

struct Node *createNode(int x) {
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));

    newNode->data = x;
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}

int height(struct Node *root) {
```

```
    if (root == NULL)
        return -1;

    int leftHeight = height(root->left);
    int rightHeight = height(root->right);

    if (leftHeight > rightHeight)
        return leftHeight + 1;
    else
        return rightHeight + 1;
}

int main() {
    // Create the binary tree
    struct Node *root = createNode(10);
    root->left = createNode(20);
    root->right = createNode(30);
    root->left->left = createNode(40);
    root->left->right = createNode(50);

    printf("Height of tree: %d\n", height(root));

    return 0;
}
```

Output :

Height of tree: 2

11. Given a binary tree, convert it into a doubly linked list keeping the same inorder traversal order.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left;
    struct Node *right;
};

struct Node *createNode(int x) {
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));

    newNode->data = x;
    newNode->left = NULL;
    newNode->right = NULL;
```

```
    return newNode;
}

void convertToDLL(struct Node *root,
                  struct Node **head,
                  struct Node **prev) {
    if (root == NULL)
        return;

    // Visit left subtree
    convertToDLL(root->left, head, prev);

    // Link current node to previous node
    if (*prev == NULL) {
        *head = root;
    } else {
        root->left = *prev;
        (*prev)->right = root;
    }

    *prev = root;

    // Visit right subtree
    convertToDLL(root->right, head, prev);
}

void displayDLL(struct Node *head) {
    struct Node *temp = head;

    printf("Inorder DLL: ");

    while (temp != NULL) {
        printf("%d", temp->data);

        if (temp->right != NULL)
            printf(" <-> ");

        temp = temp->right;
    }

    printf(" <-> NULL\n");
}

int main() {
    // Create the binary tree
    struct Node *root = createNode(10);
    root->left = createNode(20);
    root->right = createNode(30);
    root->left->left = createNode(40);
```

```
    root->left->right = createNode(50);

    struct Node *head = NULL;
    struct Node *prev = NULL;

    convertToDLL(root, &head, &prev);

    if (prev != NULL)
        prev->right = NULL;

    displayDLL(head);

    return 0;
}
```

Output :

Inorder DLL: 40 <-> 20 <-> 50 <-> 10 <-> 30 <-> NULL

12. Given a binary tree, connect the nodes that are at the same level using next-right pointers.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left;
    struct Node *right;
    struct Node *nextRight;
};

struct Node *createNode(int x) {
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));

    newNode->data = x;
    newNode->left = NULL;
    newNode->right = NULL;
    newNode->nextRight = NULL;

    return newNode;
}

void connectNextRight(struct Node *root) {
    if (root == NULL)
        return;
```

```
    struct Node *queue[100];
    int front = 0, rear = 0;

    queue[rear++] = root;

    while (front < rear) {
        int levelSize = rear - front;
        struct Node *previous = NULL;

        for (int i = 0; i < levelSize; i++) {
            struct Node *current = queue[front++];

            if (previous != NULL)
                previous->nextRight = current;

            previous = current;

            if (current->left != NULL)
                queue[rear++] = current->left;

            if (current->right != NULL)
                queue[rear++] = current->right;
        }

        // Last node on this level points to NULL
        previous->nextRight = NULL;
    }
}

void printLevels(struct Node *root) {
    if (root == NULL)
        return;

    struct Node *queue[100];
    int front = 0, rear = 0;

    queue[rear++] = root;

    while (front < rear) {
        int levelSize = rear - front;

        while (levelSize--) {
            struct Node *current = queue[front++];

            printf("%d -> ", current->data);

            if (current->nextRight != NULL)
                printf("%d", current->nextRight->data);
            else
                printf("NULL");
```

```
            printf("\n");

            if (current->left != NULL)
                queue[rear++] = current->left;

            if (current->right != NULL)
                queue[rear++] = current->right;
        }

        printf("\n");
    }
}

int main() {
    struct Node *root = createNode(10);
    root->left = createNode(20);
    root->right = createNode(30);
    root->left->left = createNode(40);
    root->left->right = createNode(50);

    connectNextRight(root);

    printf("Node -> nextRight\n");
    printLevels(root);

    return 0;
}
```

Output :

```
Node -> nextRight
10 -> NULL

20 -> 30
30 -> NULL

40 -> 50
50 -> NULL
```

13. Implement a Priority Queue using a BST and perform Insert

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
```

```
};

struct Node* insert(struct Node *root, int value) {
   if (root == NULL) {
      struct Node *newNode =
         (struct Node*)malloc(sizeof(struct Node));

      newNode->data  =  value;
      newNode->left  =  NULL;
      newNode->right = NULL;

      return newNode;
   }

   if (value < root->data)
      root->left = insert(root->left, value);
   else if (value > root->data)
      root->right = insert(root->right, value);

   return root;
}

void inorder(struct Node *root) {
   if (root != NULL) {
      inorder(root->left);
      printf("%d ", root->data);
      inorder(root->right);
   }
}

int highestPriority(struct Node *root) {
   while (root->right != NULL)
      root = root->right;

   return root->data;
}

int main() {
   struct Node *root = NULL;
   int n, value, i;

   printf("Enter number of elements: ");
   scanf("%d", &n);

   for (i = 0; i < n; i++) {
      printf("Enter priority value: ");
      scanf("%d", &value);

      root = insert(root, value);
   }
```

```
    printf("BST in sorted order: ");
    inorder(root);

    if (root != NULL)
        printf("\nHighest priority: %d\n",
              highestPriority(root));

    return 0;
}
```

Output :

```
Enter number of elements: 5
Enter priority value: 40
Enter priority value: 20
Enter priority value: 60
Enter priority value: 10
Enter priority value: 50
BST in sorted order: 10 20 40 50 60
Highest priority: 60
```

14. BST deletion

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node* insert(struct Node *root, int value) {
    if (root == NULL) {
        struct Node *newNode =
            (struct Node*)malloc(sizeof(struct Node));

        newNode->data = value;
        newNode->left = NULL;
        newNode->right = NULL;
        return newNode;
    }

    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);
```

```
    return root;
}

struct Node* findMin(struct Node *root) {
    while (root != NULL && root->left != NULL)
        root = root->left;

    return root;
}

struct Node* deleteNode(struct Node *root, int key) {
    if (root == NULL)
        return NULL;

    if (key < root->data) {
        root->left = deleteNode(root->left, key);
    }
    else if (key > root->data) {
        root->right = deleteNode(root->right, key);
    }
    else {
        // Case 1: No children
        if (root->left == NULL &&
            root->right == NULL) {
            free(root);
            return NULL;
        }

        // Case 2: Only right child
        if (root->left == NULL) {
            struct Node *temp = root->right;
            free(root);
            return temp;
        }

        // Case 2: Only left child
        if (root->right == NULL) {
            struct Node *temp = root->left;
            free(root);
            return temp;
        }

        // Case 3: Two children
        struct Node *temp = findMin(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }

    return root;
}
```

```
void inorder(struct Node *root) {
   if (root != NULL) {
      inorder(root->left);
      printf("%d ", root->data);
      inorder(root->right);
   }
}

int main() {
   struct Node *root = NULL;
   int n, value, key, i;

   printf("Enter number of nodes: ");
   scanf("%d", &n);

   for (i = 0; i < n; i++) {
      scanf("%d", &value);
      root = insert(root, value);
   }

   printf("Inorder before deletion: ");
   inorder(root);

   printf("\nEnter value to delete: ");
   scanf("%d", &key);

   root = deleteNode(root, key);

   printf("Inorder after deletion: ");
   inorder(root);
   printf("\n");

   return 0;
}
```

Output :

```
Enter number of nodes: 7
50 30 70 20 40 60 80
Enter value to delete: 50
Inorder before deletion: 20 30 40 50 60 70 80
Inorder after deletion: 20 30 40 60 70 80
```

15. C program to convert postfix expression into an expression tree

```
#include <stdio.h>
#include <stdlib.h>
```

```
#include <ctype.h>
#include <string.h>

struct Node {
   char data;
   struct Node *left, *right;
};

struct Node* createNode(char value) {
   struct Node *newNode =
      (struct Node*)malloc(sizeof(struct Node));

   newNode->data  =  value;
   newNode->left  =  NULL;
   newNode->right = NULL;

   return newNode;
}

void preorder(struct Node *root) {
   if (root != NULL) {
      printf("%c ", root->data);
      preorder(root->left);
      preorder(root->right);
   }
}

void inorder(struct Node *root) {
   if (root != NULL) {
      if (root->left != NULL)
         printf("(");

      inorder(root->left);
      printf("%c", root->data);
      inorder(root->right);

      if (root->right != NULL)
         printf(")");
   }
}

int main() {
   char postfix[100];
   struct Node *stack[100];
   int top = -1;
   int i;

   printf("Enter postfix expression: ");
   scanf("%99s", postfix);
```

```
    for (i = 0; postfix[i] != '\0'; i++) {
        char ch = postfix[i];

        if (isalnum((unsigned char)ch)) {
            stack[++top] = createNode(ch);
        }
        else {
            if (top < 1) {
                printf("Invalid postfix expression\n");
                return 1;
            }

            struct Node *newNode = createNode(ch);

            newNode->right = stack[top--];
            newNode->left = stack[top--];

            stack[++top] = newNode;
        }
    }

    if (top != 0) {
        printf("Invalid postfix expression\n");
        return 1;
    }

    struct Node *root = stack[top];

    printf("Inorder expression: ");
    inorder(root);

    printf("\nPreorder expression: ");
    preorder(root);

    printf("\n");

    return 0;
}
```

Output :

```
Enter postfix expression: ab-cd*+
Inorder expression: (a-b)+(c*d)
Preorder expression: + - a b * c d
```

16 .Construct a BST and display its Inorder, Preorder and Postorder traversals.

```
#include <stdio.h>
```

```
#include <stdlib.h>

struct Node {
   int data;
   struct Node *left, *right;
};

struct Node* insert(struct Node *root, int value) {
   if (root == NULL) {
      struct Node *newNode =
         (struct Node*)malloc(sizeof(struct Node));

      newNode->data  =  value;
      newNode->left  =  NULL;
      newNode->right = NULL;

      return newNode;
   }

   if (value < root->data)
      root->left = insert(root->left, value);
   else if (value > root->data)
      root->right = insert(root->right, value);

   return root;
}

void inorder(struct Node *root) {
   if (root != NULL) {
      inorder(root->left);
      printf("%d ", root->data);
      inorder(root->right);
   }
}

void preorder(struct Node *root) {
   if (root != NULL) {
      printf("%d ", root->data);
      preorder(root->left);
      preorder(root->right);
   }
}

void postorder(struct Node *root) {
   if (root != NULL) {
      postorder(root->left);
      postorder(root->right);
      printf("%d ", root->data);
   }
}
```

```
int main() {
    struct Node *root = NULL;
    int n, value, i;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter values: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &value);
        root = insert(root, value);
    }

    printf("Inorder: ");
    inorder(root);

    printf("\nPreorder: ");
    preorder(root);

    printf("\nPostorder: ");
    postorder(root);

    printf("\n");
    return 0;
}
```

Output :

```
Enter number of nodes: 5
Enter values: 40 20 60 10 30
Inorder: 10 20 30 40 60
Preorder: 40 20 10 30 60
Postorder: 10 30 20 60 40
```

17.Write a C program to implement Heap Sort

```
#include <stdio.h>

void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    int temp;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
```

```
        largest = right;

    if (largest != i) {
        temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {
    int i, temp;

    // Build max heap
    for (i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Move maximum to the end repeatedly
    for (i = n - 1; i > 0; i--) {
        temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        heapify(arr, i, 0);
    }
}

int main() {
    int arr[100], n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    heapSort(arr, n);

    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);

    printf("\n");
    return 0;
}
```

Output :

Enter number of elements: 6
Enter elements: 50 20 30 15 22 32
Sorted array: 15 20 22 30 32 50